# Web-Services Projekt

Kartenspiel Skyjo

| | |
|---|---|
| **Kurs:** | Web-Services |
| **Professor:** | Prof. Dr. Alexander Auch |
| **Gruppenmitglieder:** | Tobias Gramlich, Mona Ittlinger, Melina Klein |
| **Datum:** | 29. März 2026 |

# Inhaltsverzeichnis

**Abbildungsverzeichnis** **1**

**1 Projektidee und Ablauf** **2**
1.1 Grundidee des Projekts “Skyjo” . . . 2
1.2 Projektplan . . . 3

**2 BPMN-Modelle** **4**
2.1 Benutzerverwaltungs-Service . . . 4
2.1.1 Registrierung . . . 4
2.1.2 Aktivierung . . . 4
2.1.3 Authentication . . . 5
2.2 Matchmaking-Service . . . 6
2.2.1 Verbindung und Login . . . 6
2.2.2 Room Interaction . . . 6
2.3 Color-Weather-API . . . 7
2.4 Skyjo-Logik . . . 7
2.4.1 Spielablauf . . . 7
2.4.2 Spielrunde . . . 8
2.4.3 Spielzug . . . 9

**3 Implementierung** **10**
3.1 Benutzerverwaltung . . . 10
3.1.1 Endpunkte . . . 10
3.2 Matchmaking . . . 12
3.2.1 WebSocket-Anfragen . . . 12
3.3 Color-Weather-API . . . 13
3.3.1 Datenquellen und Klassifizierung . . . 13
3.3.2 Datenstruktur und UI-Integration . . . 13
3.4 skyjo-logic . . . 14
3.4.1 Datenbankschema . . . 15
3.4.2 Spielablauf und Kommunikation . . . 16
3.5 Weitere Hinweise zur Implementierung . . . 18

**4 Lessons Learnt** **19**
4.1 Zeitmanagement . . . 19
4.2 Kommunikation . . . 19
4.3 Auswahl der Technologien . . . 19

# Abbildungsverzeichnis

2.1 Registrierung . . . . . 4
2.2 Aktivierung . . . . . 4
2.3 Login . . . . . 5
2.4 Authentication . . . . . 5
2.5 Verbindung und Login . . . . . 6
2.6 Room Interaction . . . . . 6
2.7 Color-Weather-API . . . . . 7
2.8 Spielablauf . . . . . 7
2.9 Spielrunde . . . . . 8
2.10 Spielzug . . . . . 9

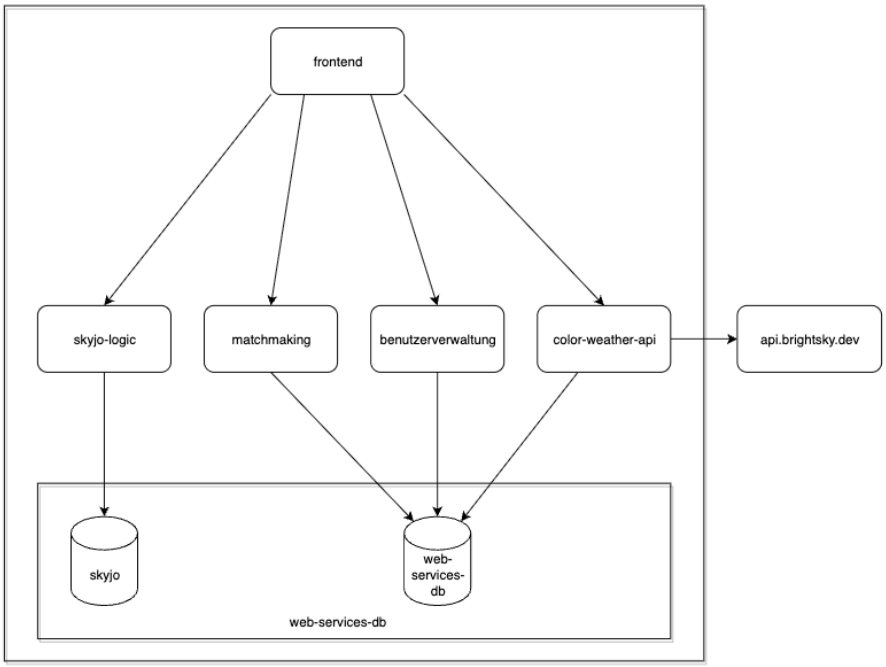
3.1 Datenbankschema der Skyjo-Spiellogik. . . . . 15

# 1 Projektidee und Ablauf

## 1.1 Grundidee des Projekts “Skyjo”

Skyjo ist ein zugängliches Kartenspiel für zwei bis acht Personen, das 2019 von Alexander Schwarzkopf entwickelt wurde. Ziel des Spiels ist es, am Ende einer Runde möglichst wenige Punkte auf der Hand zu halten. Jeder Spieler erhält zu Beginn zwölf verdeckte Karten, die er im Laufe des Spiels schrittweise aufdecken und durch Karten aus dem Zieh- oder Ablagestapel ersetzen kann. Eine Runde endet, sobald ein Spieler alle seine Karten aufgedeckt hat. Anschließend werden die Punkte aller Spieler summiert. Das Spiel endet, wenn ein Spieler 100 oder mehr Punkte erreicht - der Spieler mit den wenigsten Gesamtpunkten gewinnt. Der Link zu einer Spielerklärung als YouTube-Video befindet sich im Repository des Projekts.

Ziel des Projekts war es, eine Variation dieses Kartenspiels als verteiltes Online-System mithilfe von Web-Services zu realisieren. Die Umsetzung erfolgte als Microservice-Architektur, bestehend aus vier eigenständigen Services: einem Benutzerverwaltungs-Service, einem Matchmaking-Service, einer Color-Weather-API sowie dem eigentlichen Skyjo-Logik-Service. Die Architektur war dabei durch die Aufgabenstellung vorgegeben. Die Implementierung unterstützt Spielrunden mit zwei bis vier Spielern.



## 1.2 Projektplan

Das Projekt startete zu Beginn des vierten Semesters in der zweiten Januarwoche mit einem Brainstorming-Termin, in dem sich die Idee, Skyjo als Web-Service zu implementieren, durchsetzte. Im anschließenden Kickoff-Meeting wurden ein Architekturmodell sowie erste BPMN-Modelle erarbeitet.
Der ursprüngliche Zeitplan sah vor, die vollständige Implementierung aller Services bis zum 08. März abzuschließen, sodass die verbleibende Zeit bis zur Präsentation am 12. März ausschließlich für die Ausarbeitung der Präsentation und der Dokumentation genutzt werden könnte. Dieses Ziel konnte jedoch nicht vollständig erreicht werden. Ursache waren Build-Fehler im Gradle-Projekt, die auf eine fehlerhafte Konfiguration des Datenbanklogins zurückzuführen waren und einen erheblichen Zeitaufwand bei der Fehlersuche verursachten. Bei der Präsentation am 12. März wurden daher alle Services mit Ausnahme der Skyjo-Logik vollständig vorgestellt; für diesen Service konnte lediglich ein Zwischenstand präsentiert werden.
In den zwei Wochen nach der Präsentation wurden die verbleibenden Aufgaben abgeschlossen: Die Skyjo-Logik wurde fertiggestellt, das Frontend für das Spiel implementiert sowie die Projektdokumentation vervollständigt.
Die Implementierung erfolgte sowohl in Einzel- als auch in Teamarbeit. Die Wahl der eingesetzten Technologien und Programmiersprachen orientierte sich dabei bewusst an den individuellen Stärken der Teammitglieder. Als besonderer technischer Exkurs wurde das Framework Quarkus eingesetzt, das auf Empfehlung des Dozenten hin erprobt wurde und an bereits vorhandene Java-Kenntnisse im Team anknüpfte.

| Name | Aufgaben |
|---|---|
| Tobias Gramlich | Benutzerverwaltungsservice, Matchmaking, Skyjo-Logic, BPMN |
| Mona Ittlinger | Matchmaking, Frontend, Dokumentation, Präsentation, BPMN |
| Melina Klein | Skyjo-Logic, Color-Weather-API, Dokumentation, BPMN |

# 2 BPMN-Modelle

## 2.1 Benutzerverwaltungs-Service

### 2.1.1 Registrierung

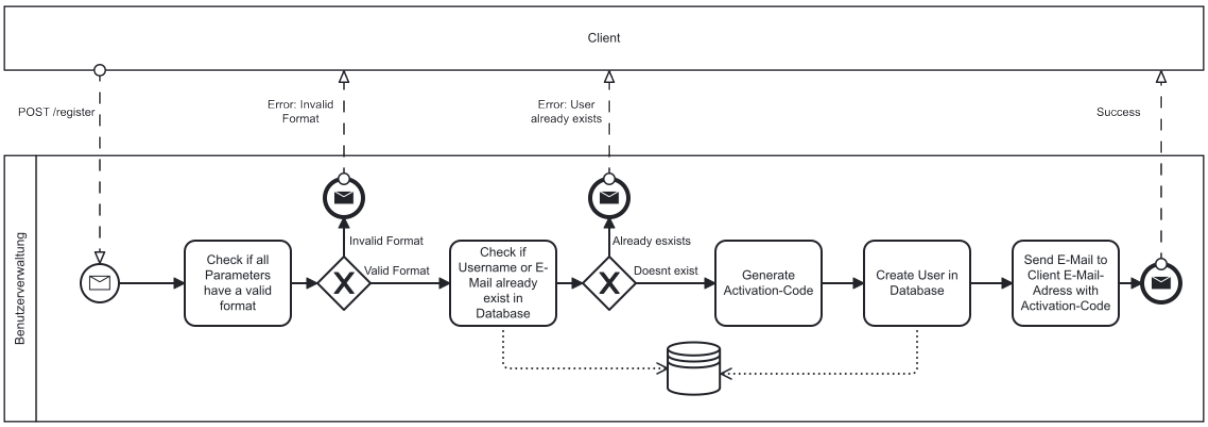



Abbildung 2.1: Registrierung

### 2.1.2 Aktivierung

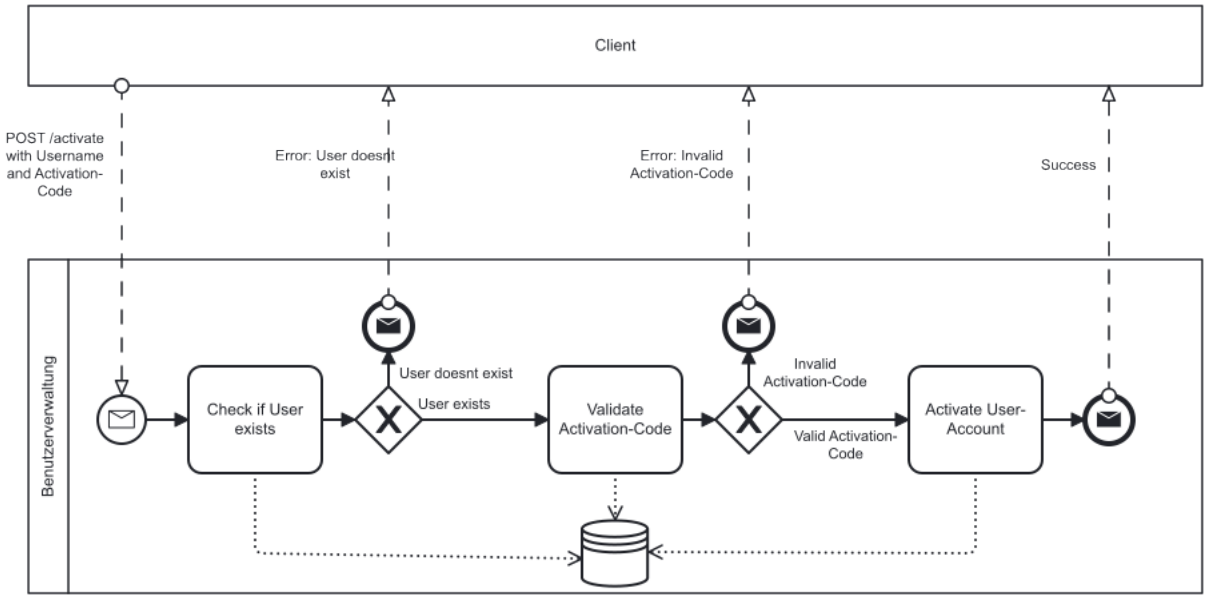



Abbildung 2.2: Aktivierung

### Login

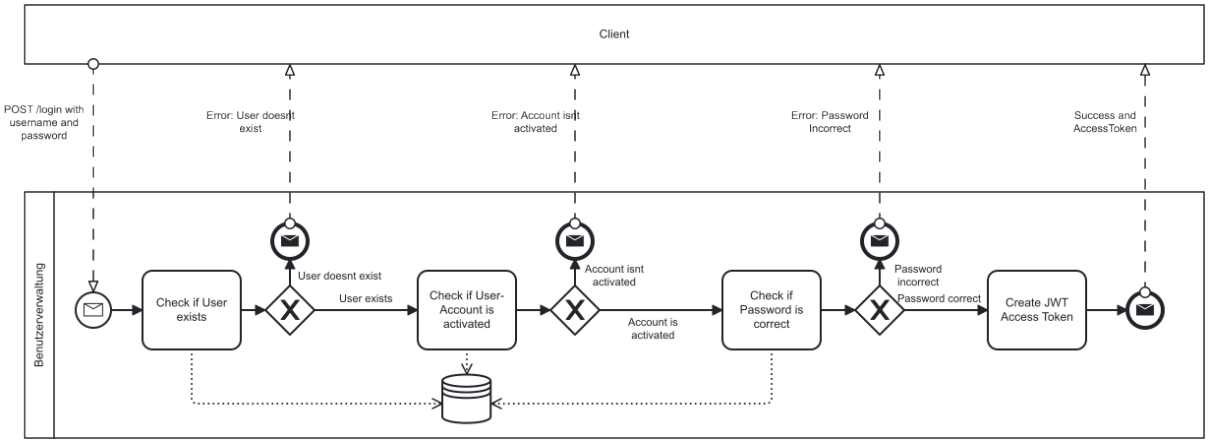



Abbildung 2.3: Login

## 2.1.3 Authentication

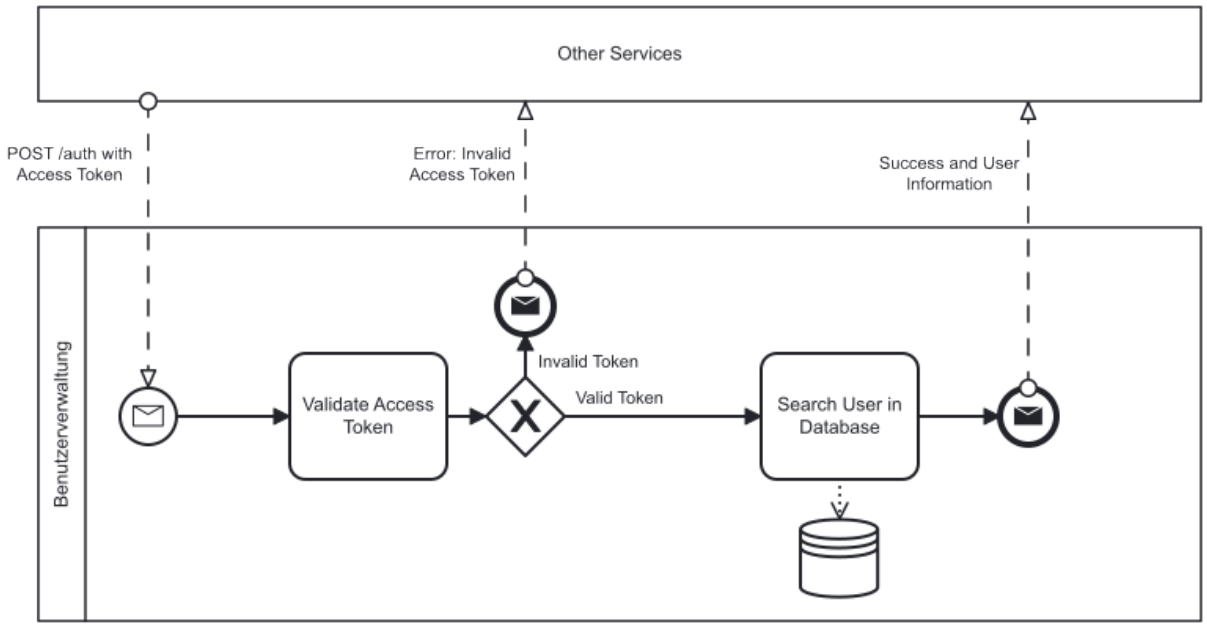



Abbildung 2.4: Authentication

## 2.2 Matchmaking-Service

### 2.2.1 Verbindung und Login

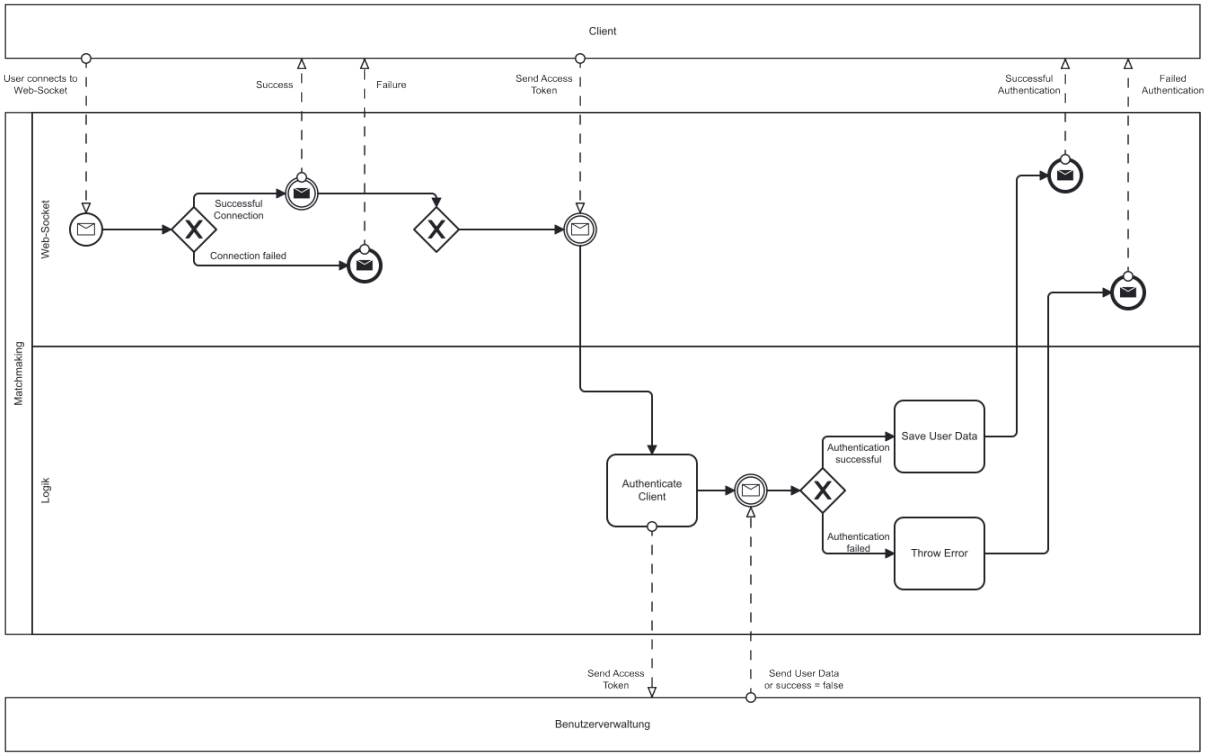



Abbildung 2.5: Verbindung und Login

### 2.2.2 Room Interaction

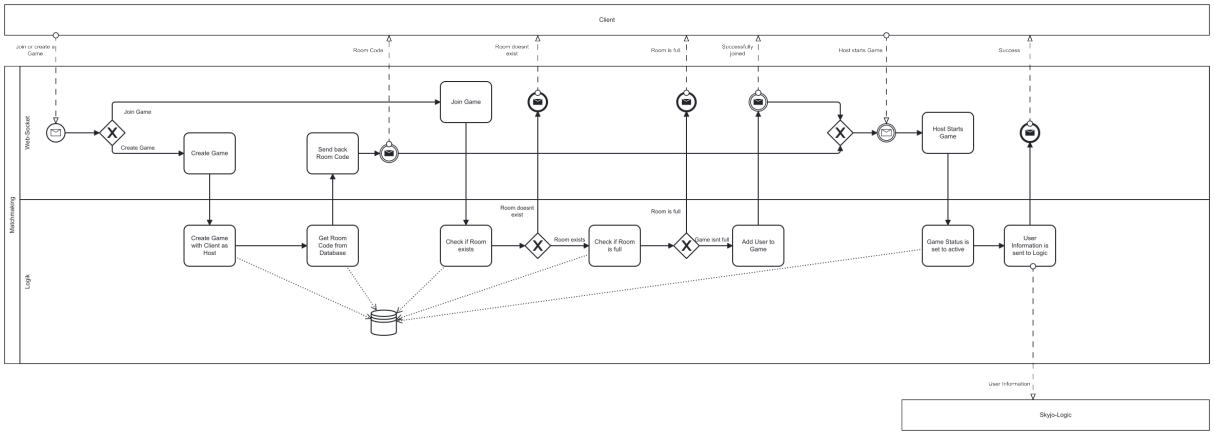



Abbildung 2.6: Room Interaction

## 2.3 Color-Weather-API

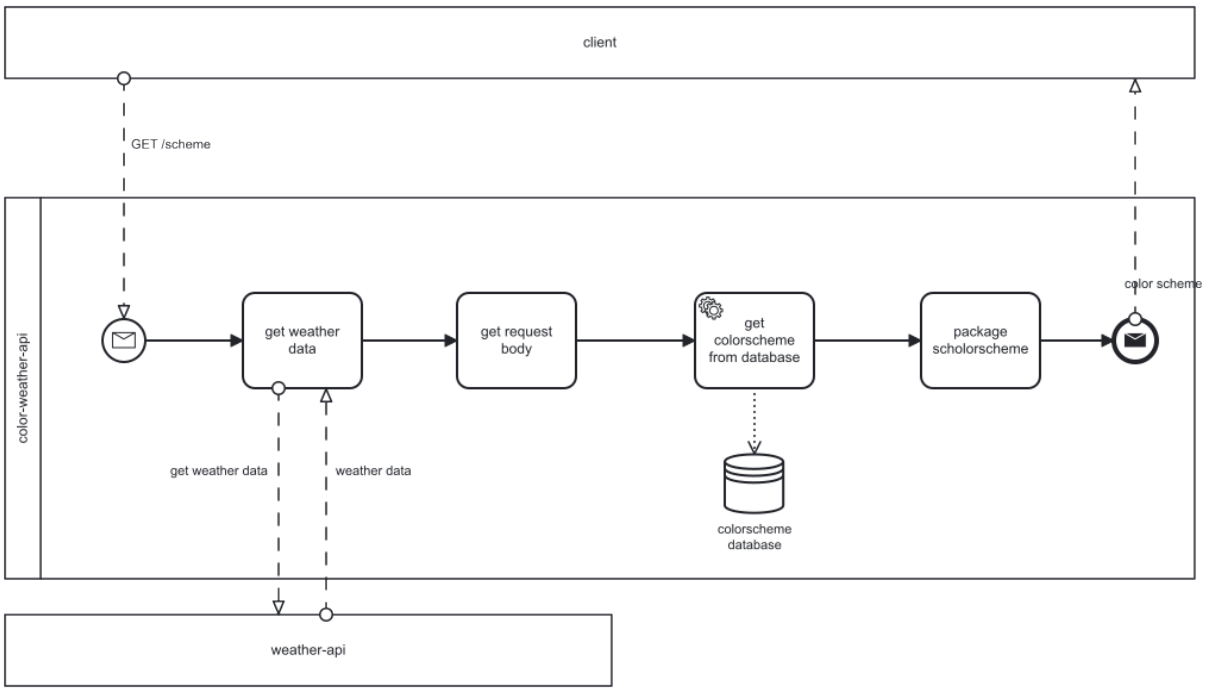



Abbildung 2.7: Color-Weather-API

## 2.4 Skyjo-Logik

### 2.4.1 Spielablauf

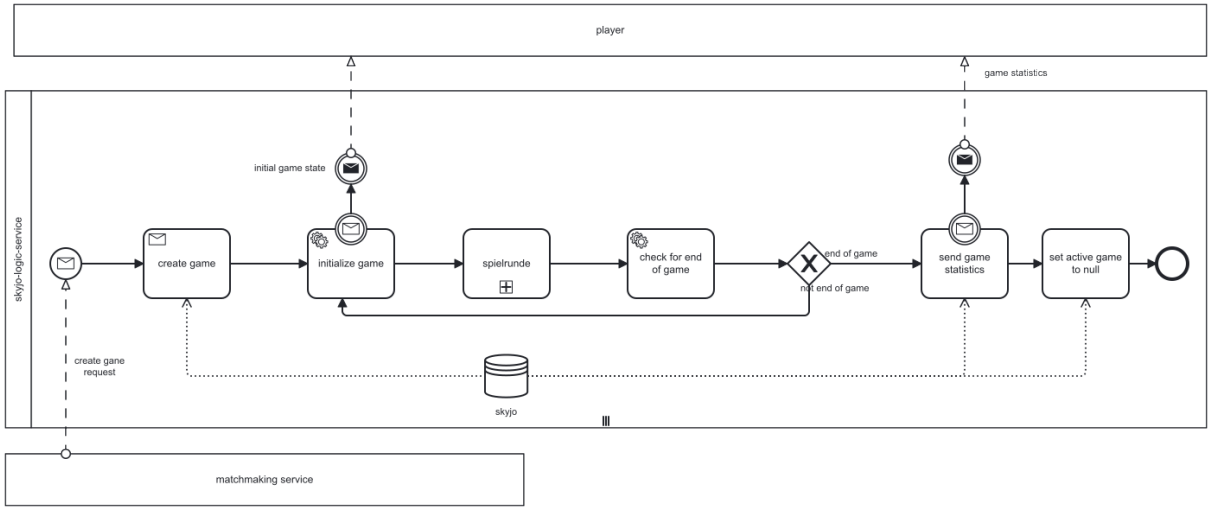



Abbildung 2.8: Spielablauf

### 2.4.2 Spielrunde

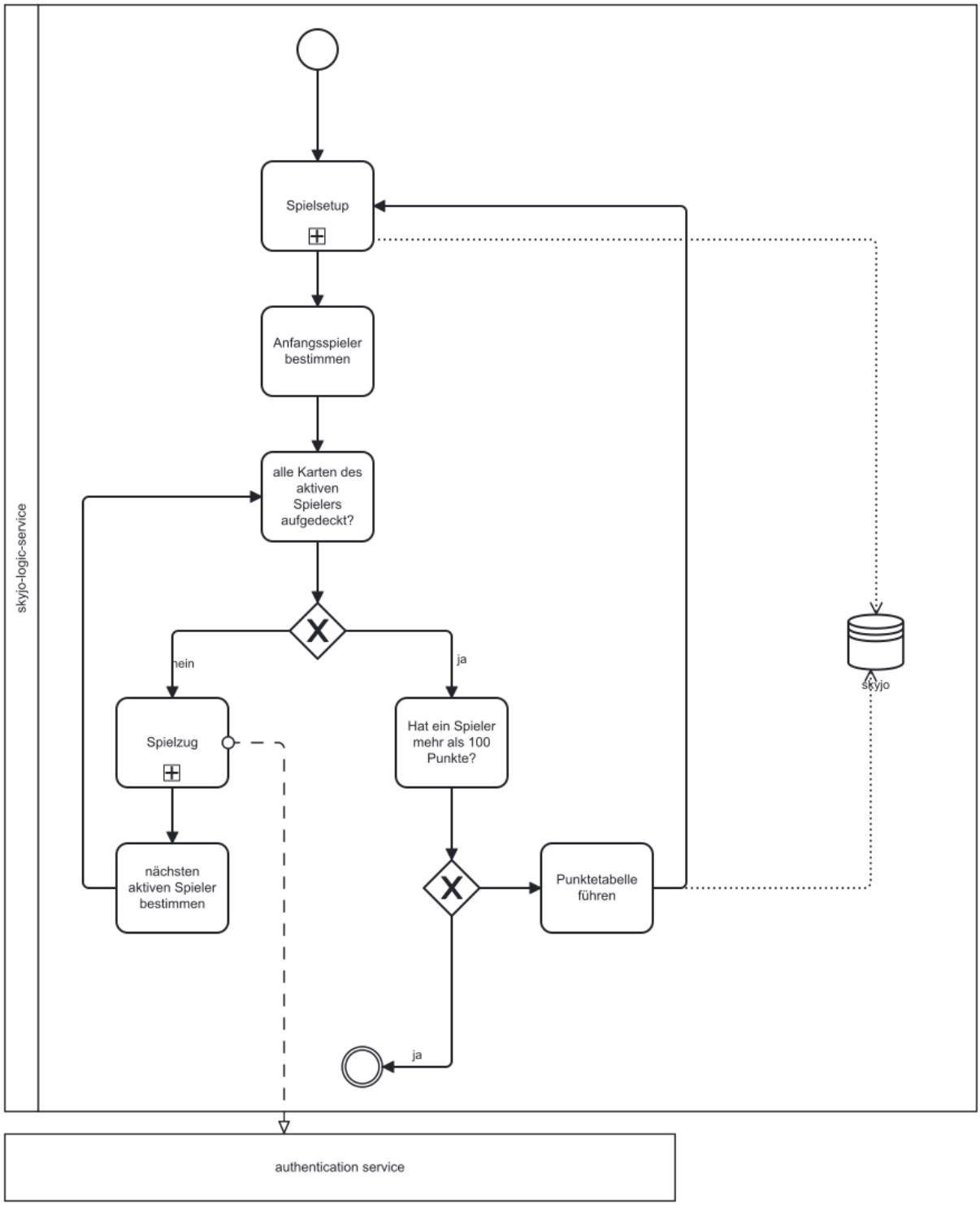



Abbildung 2.9: Spielrunde

### 2.4.3 Spielzug

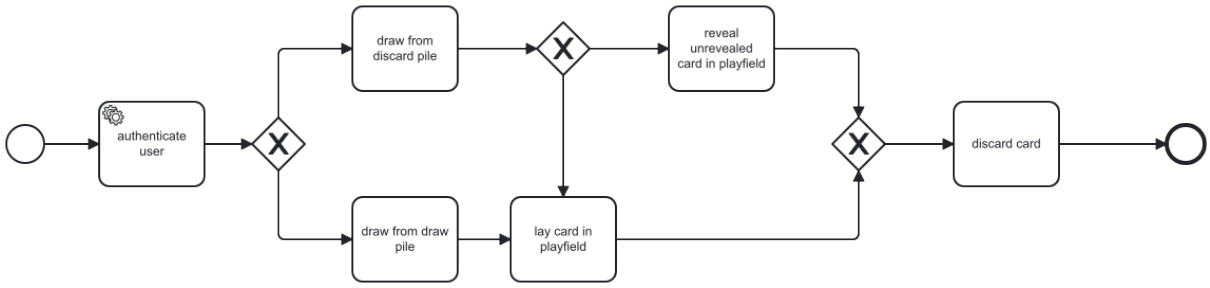



Abbildung 2.10: Spielzug

# 3 Implementierung

## 3.1 Benutzerverwaltung

Die Benutzerverwaltung wurde als eigener Web-Service in JavaScript mit Hilfe von Node.js umgesetzt. Ziel war es, einen Web-Service zu implementieren, bei welchem sich Nutzer registrieren und anmelden können und der diese Nutzer dann bei Aktionen authentifiziert. Für die Bereitstellung der Endpunkte wurden Express und CORS benutzt. Als Datenbank wurde MariaDB mit Sequelize als ORM gewählt.

### 3.1.1 Endpunkte

**Registrierung:** Zunächst gibt es einen Endpunkt, bei dem sich Nutzer registrieren können. Der Ablauf wird in Abbildung 2.1 dargestellt. Dabei müssen sie sich einen Benutzernamen und ein Passwort ausdenken sowie eine E-Mail-Adresse angeben. Das Backend überprüft, ob diese Daten gültige Formate haben und ob eventuell bereits jemand unter diesem Namen registriert ist. Danach wird ein neuer Nutzer in der Datenbank erstellt, ein Aktivierungscode generiert und an die E-Mail-Adresse versendet. Der E-Mail-Versand geschieht über SMTP mithilfe des NodeMailer-Moduls. Das Passwort wird mithilfe von bcrypt verschlüsselt, sodass es nicht im Klartext gespeichert wird:

```
const hash = await bcrypt.hash(password, 10);
await Users.create({
    username: username,
    password: hash,
    email: email,
    activationcode: activationcode
});
```

**E-Mail erneut senden:** Sollte der E-Mail-Versand fehlschlagen oder die E-Mail nicht ankommen, kann über diesen Endpunkt erneut eine E-Mail mit Aktivierungscode angefordert werden. Dafür müssen Benutzername und Passwort angegeben werden.

**Aktivierung:** Um sich einloggen zu können, muss das Benutzerkonto zunächst über diesen Endpunkt aktiviert werden. Dazu sendet der Nutzer seinen Benutzernamen und den per E-Mail erhaltenen Aktivierungscode. Nach erfolgreicher Validierung wird das Konto in der Datenbank als aktiviert hinterlegt. Das BPMN-Modell zur Aktivierung ist in Abschnitt 2.2 zu finden.

**Login:** Über den Login-Endpunkt kann sich der Nutzer auf seinem Konto anmelden, in Abbildung 2.3 ist dieser grafisch dargestellt. Das Backend überprüft Passwort und Aktivierungsstatus und erstellt anschließend ein JSON Web Token (JWT), das als Access-Token zurückgesendet wird. Das JWT hat eine standardmäßige Gültigkeitsdauer von 7 Tagen:

```
const accessToken = sign(
    {
        username: user.username,
        id: user.id,
        email: user.email
    },
    process.env.JWT_SECRET,
    {
        expiresIn: process.env.JWT_EXPIRE,
        issuer: process.env.JWT_ISSUER,
        algorithm: process.env.JWT_ALGORITHM
    }
);
```

**Auth:** Über den Auth-Endpunkt können andere Services überprüfen, ob ein Benutzer authentifiziert ist. Dafür wird das JWT des Benutzers übermittelt, validiert und anschließend werden Benutzer-ID, Benutzername und E-Mail zurückgegeben. Der Ablauf des Authentifizierungprozesses ist in Abbildung 2.4 zu sehen.

**Benutzerinformationen ändern:** Über zwei separate Endpunkte können Benutzername und Passwort geändert werden. In beiden Fällen müssen aktuelle Zugangsdaten sowie der neue Wert angegeben werden.

**Konto löschen:** Über diesen Endpunkt kann ein Benutzerkonto unwiderruflich gelöscht werden. Das JWT wird dabei validiert, bevor der Datensatz aus der Datenbank entfernt wird.

*Hinweis:* Der E-Mail-Versand wurde zwar implementiert und getestet, in der finalen Version jedoch aufgrund der Ausführung auf einem DHBW-Rechner deaktiviert. Dies wurde vorab abgesprochen.

## 3.2 Matchmaking

Der Matchmaking-Service wurde ebenfalls in JavaScript mit Node.js implementiert. Ziel war es, dass Benutzer Spielräume erstellen, diesen beitreten und das Spiel starten können, sobald der Raum voll ist. Dafür wurde ein WebSocket-Server (WSS) auf Basis des Moduls `ws` verwendet. Benutzer verbinden sich mit dem WSS und stellen spezielle Anfragen, um mit dem System zu interagieren. Der WebSocket sendet Antworten und Raumaktualisierungen zurück:

```
function broadcastToRoom(store, roomCode, obj) {
    const code = String(roomCode);
    const entry = store.roomSockets.get(code);
    if (!entry) return;
    for (const ws of entry.sockets) safeSend(ws, obj);
}
```

### 3.2.1 WebSocket-Anfragen

**Authenticate:** Bevor ein Nutzer andere Aktionen ausführen kann, muss er sich authentifizieren. Er sendet ein Authentication-Request mit seinem JWT, das an die Benutzerverwaltung weitergeleitet und dort validiert wird.

**Raum erstellen:** Nach der Authentifizierung kann ein Nutzer einen Raum erstellen. Dieser wird in der Datenbank gespeichert; die generierte Datenbank-ID dient als Raumcode. Die WebSocket-Verbindung wird automatisch in einen `RoomStore` geladen, der für die Nachrichtenvermittlung innerhalb des Raums zuständig ist.

**Raum beitreten:** Durch Angabe des Raumcodes kann ein Nutzer einem Raum beitreten. Das Backend prüft, ob der Nutzer bereits in einem anderen Raum ist oder ob der Raum voll ist, und speichert den Nutzer anschließend in der Datenbank. Die übrigen Spieler erhalten eine entsprechende Benachrichtigung.

**Raum verlassen:** Spieler können Räume verlassen. Das Backend entfernt sie aus dem Raum in der Datenbank und aus dem `RoomStore`. Die anderen Spieler werden benachrichtigt.

**Spiel starten:** Der Host des Raumes kann bei 2–4 Spielern das Spiel starten. Die Spieler-IDs werden aus der Datenbank geladen und an den Skyjo-Logik-Service übergeben, der ab diesem Zeitpunkt die Spielsteuerung übernimmt.

## 3.3 Color-Weather-API

Die technische Umsetzung der Color-Weather-API erfolgte mit dem FastAPI-Framework in Python. Ziel war es, eine performante Schnittstelle zu schaffen, die basierend auf Echtzeit-Wetterdaten und zeitlichem Kontext ein harmonisches UI-Farbschema generiert.

### 3.3.1 Datenquellen und Klassifizierung

Als primäre Datenquelle dient die Brightsky-API. Diese Open-Source-Alternative wurde aufgrund ihrer hohen Verfügbarkeit und des Verzichts auf API-Keys gewählt. Die API-Logik transformiert die Wetterdaten und Zeitstempel in drei Dimensionen:

1. Wetterlage (5 Kategorien): Sunny, Cloudy, Storm, Snow, Rainy.
2. Tagesphase (3 Kategorien): Morning, Midday, Night.
3. Wochenrhythmus (3 Kategorien): Differenzierung zwischen Arbeitswoche (ruhige Töne), Freitag (lebendige Akzente) und Wochenende (entspannte Atmosphäre).

Durch die Kombination dieser Faktoren ergeben sich insgesamt 60 distinkte Farbkombinationen. Diese Kombinationen werden in einer MariaDB-Datenbank gespeichert, welche bewusst denormalisiert wurde, um einfache Queries nutzen zu können.

### 3.3.2 Datenstruktur und UI-Integration

Die Rückgabe der API erfolgt über ein validiertes Pydantic-Modell, um Konsistenz im Frontend sicherzustellen:

```python
class ColorResponse(BaseModel):
    background: Color        # Basis der Anwendung (idR dunkler/
        entsaettigt)
    surface: Color           # Karten, Container, Eingabefelder
    primary_color: Color     # Fokus-Elemente, Buttons, Ueberschriften
    secondary_color: Color   # Akzente und ergaenzende UI-Elemente
```

Listing 3.1: Pydantic-Modell für Farbschema

Die gestalterische Logik folgt dem Prinzip der Barrierefreiheit und schafft eine visuelle Hierarchie. Für eine klare räumliche Trennung wird der `background` bewusst etwas dunkler und entsättigter gewählt. Dadurch wird ein subtiler Kontrast zu `surface`-Elementen, wie Cards oder andere Container, geschaffen.
`primary_color` und `secondary_color` sind farblich eng aufeinander abgestimmt und navigieren den Nutzer durch die Oberfläche.
Durch diese Abstimmung der einzelnen Farben wirken die Schemen intuitiv und strukturiert.

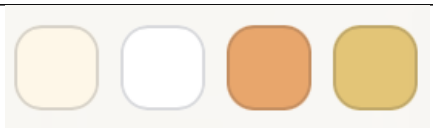
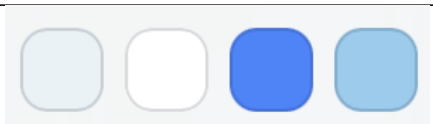
| `time_of_day` | `day_type` | `weather` | **Farbschema** |
|---|---|---|---|
| Morning | Weekday | Sunny |  |
| Morning | Friday | Rainy |  |
| Night | Weekend | Storm | |

Tabelle 3.1: Beispiel-Schemata für verschiedene Eingaben

## 3.4 skyjo-logic

Der Webservice wurde in Java mit dem Quarkus-Framework implementiert. Hierbei wurde eine Kombination aus einer REST-API und einem WebSocket implementiert, die es ermöglicht, idempotente REST-API-Calls und echtzeitsensible UI-Updates zu generieren. So kann ein reibungsloser Spielablauf gewährleistet werden.

### 3.4.1 Datenbankschema

Zur Datenpersistenz wurde eine MariaDB-Datenbank eingesetzt. Um eine strukturierte Versionsverwaltung des Schemas zu gewährleisten, wurde das Framework Flyway verwendet. Flyway ermöglicht es, Schemaänderungen mithilfe von SQL-Migrationen versioniert zu verwalten und konsistent über verschiedene Umgebungen hinweg auszurollen. Das resultierende Datenbankschema der Spiellogik ist in Abbildung 3.1 dargestellt.

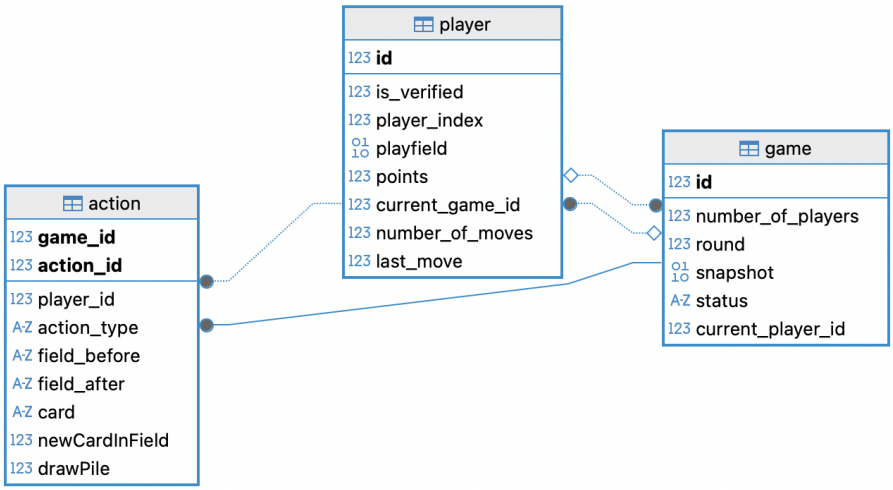



Abbildung 3.1: Datenbankschema der Skyjo-Spiellogik.

Die Tabelle `game` repräsentiert eine einzelne Spielsitzung. Neben der Anzahl der Spieler (`number_of_players`) und der aktuellen Runde (`round`) wird ein `snapshot` gespeichert, der den vollständigen Spielzustand als serialisiertes Objekt enthält - etwa den Zieh- und Ablagestapel. Das Feld `status` hält den aktuellen Spielstatus (z. B. `WAITING`, `RUNNING`, `FINISHED`) fest. Über `current_player_id` wird referenziert, welcher Spieler gerade am Zug ist.

Die Tabelle `player` repräsentiert einen Spieler innerhalb einer Partie. Der Fremdschlüssel `current_game_id` verknüpft jeden Spieler mit genau einem Spiel. Das Feld `player_index` bestimmt die Sitzreihenfolge. Das `playfield` wird, analog zum `snapshot` in `game`, als serialisiertes Objekt gespeichert und enthält das individuelle Kartenfeld des Spielers. Zusätzlich werden die aktuellen Punkte (`points`), die Anzahl der gespielten Züge (`number_of_moves`) sowie der letzte Zug (`last_move`) festgehalten. Das Flag `is_verified` zeigt an, ob der Spieler seine Teilnahme bestätigt hat.

Die Tabelle `action` protokolliert jeden einzelnen Spielzug und bildet damit das vollständige Spielprotokoll. Der zusammengesetzte Primärschlüssel aus `game_id` und `action_id` stellt sicher, dass Aktionen eindeutig einer Partie zugeordnet sind. Über `player_id` ist jede Aktion einem Spieler zugewiesen. Der Typ des Zuges wird in `action_type` gespeichert (z. B.

Karte ziehen, tauschen). Die Felder `field_before` und `field_after` dokumentieren den Zustand des Spielfelds vor und nach dem Zug, was eine vollständige Nachvollziehbarkeit ermöglicht. Die gespielte Karte (`card`), die neue Karte im Spielfeld (`newCardInField`) sowie der verbleibende Ziehstapel (`drawPile`) vervollständigen den Aktionsdatensatz.

Zwischen `game` und `player` besteht eine 1:n-Beziehung: ein Spiel hat mehrere Spieler, jeder Spieler gehört zu genau einem Spiel. Die Relation `current_player_id` in `game` verweist zusätzlich auf den aktuell aktiven Spieler (optionale 1:1-Beziehung). Die Tabelle `action` ist über `game_id` und `player_id` mit beiden anderen Tabellen verknüpft, sodass jeder Zug eindeutig einer Partie und einem Spieler zugeordnet werden kann.

An verschiedenen Stellen wurde die Datenbank bewusst denormalisiert. Beispielsweise wird der in `snapshot` gespeicherte Spielstand als serialisiertes Objekt abgelegt, da eine normalisierte Darstellung bei jedem Lesezugriff aufwändige Joins erfordern würde. Um dennoch die Konsistenz des Systems zu wahren, lassen sich durch die Einträge in `action` alle Spielzüge konsistent nachvollziehen und bei Bedarf rückgängig machen, da jede Action das Spielfeld des betreffenden Spielers sowohl vor als auch nach einem validen Zug speichert.

### 3.4.2 Spielablauf und Kommunikation

Der folgende Abschnitt beschreibt den Spielablauf, wie er innerhalb des `skyjo-logic`-Services umgesetzt wurde. Das BPMN-Modell in Abschnitt 2.4 kann ergänzend zur Veranschaulichung herangezogen werden.

**Spielinitialisierung** Ein Spiel wird über den Endpunkt `/setUpGame` erstellt, wobei die IDs der teilnehmenden Spieler übergeben werden. Im Rahmen des Game-Setups werden die Spieler initialisiert, die Karten ausgeteilt und der Startspieler bestimmt. Anschließend wird der vollständige Spielzustand persistiert. Die eindeutige Spiel-ID wird nicht vom Aufrufer vergeben, sondern von der Datenbank generiert, wodurch Kollisionen zuverlässig vermieden werden. Dafür wurde die Spalte `id` ist mit dem Constraint `AUTO_INCREMENT` versehen.Gleichzeitig wird im Rahmen der Spielinitialisierung die WebSocket-Verbindung aufgebaut, über die alle späteren Echtzeit-Updates an die Clients übermittelt werden. Die Weiterleitung zum `skyjo-logic`-Service erfolgt dabei durch den Matchmaking-Service.

**Authentifizierung und Autorisierung** Ein Spieler authentifiziert sich mittels JWT-Token, das bei jeder Anfrage mitgesendet und vom Benutzerverwaltungsservice validiert wird. Nur erfolgreich verifizierte Spieler dürfen aktiv am Spiel teilnehmen. Versucht ein nicht verifizierter Spieler einen Endpunkt aufzurufen, wird die Anfrage mit dem HTTP-Statuscode `401 Unauthorized` abgewiesen.

**Spielzug** Ein Spiel besteht aus mehreren Runden, in denen die Spieler in einer festgelegten Reihenfolge ihre Züge ausführen. Der genaue Ablauf eines Spielzugs ist im Diagramm *Spielzug* in Abschnitt 2.4 dargestellt. Die oberste Karte des Ablagestapels ist jederzeit einsehbar; die Karte vom Ziehstapel kann über den Endpunkt `/getCard` abgerufen werden. Eine Karte des eigenen Spielfelds wird, sofern erforderlich, erst beim Abschluss des Zuges aufgedeckt.

Vor der eigentlichen Ausführung eines Zuges wird eine `action` angelegt, die `game_id`, `action_id` und `player_id` enthält. Diese kann nur dann mit den Zugdaten befüllt werden, wenn der entsprechende Spieler aktuell an der Reihe ist. Dadurch wird sichergestellt, dass keine zwei Spieler gleichzeitig einen Zug ausführen können und die REST-Endpunkte idempotent bleiben - ein mehrfach gesendeter Request hat keine ungewollten Seiteneffekte, da die Aktion bereits angelegt ist und nur einmal befüllt werden kann.

**Rundenende** Eine Runde endet, sobald ein Spieler alle seine Karten aufgedeckt hat. Danach erhält jeder verbleibende Spieler noch genau einen weiteren Zug. Die Überprüfung, ob es sich um den letzten Zug handelt, erfolgt über die Methode `isLastMove()` in der Klasse `Game`:

```
public boolean isLastMove(Long playerId) {
    // check if current player is actually playing
    if (!playerId.equals(getCurrentPlayer().getId())) {
        throw new IllegalArgumentException(
            "Player␣id␣is␣not␣the␣same␣as␣current␣player.");
    }
    // check if they already played their last move
    if (getCurrentPlayer().getLastMoveDone()) {
        throw new IllegalArgumentException(
            "Player␣last␣move␣is␣already␣done");
    }
    // last move if player before has done their last move
    assert players != null;
    return players.get(
        Math.floorMod(currentPlayerIndex - 1, players.size())
    ).getLastMoveDone();
}
```

Sobald ein Spieler alle Karten aufgedeckt hat, wird sein Attribut `lastMoveDone` auf `true` gesetzt. Der jeweils nachfolgende Spieler prüft beim Ausführen seines Zuges, ob der Vorgänger seinen letzten Zug bereits abgeschlossen hat, und setzt sein eigenes Attribut entsprechend. Haben alle Spieler ihren letzten Zug absolviert, wird die Runde beendet und das Rundenergebnis als JSON-Struktur über den WebSocket an alle Clients übermittelt.

**Spielende und UI-Updates** Nach jeder Runde wird geprüft, ob das Spiel beendet ist. Dies ist der Fall, wenn mindestens ein Spieler die Punktegrenze von 100 erreicht oder überschritten hat. In diesem Fall wird anstelle des Rundenergebnisses das Endergebnis als JSON-Struktur über den WebSocket verschickt. Ist weder das Rundenende noch das Spielende eingetreten, wird die Aktion verarbeitet und ein UI-Update innerhalb der laufenden Runde an die Clients gesendet.

Alle WebSocket-Nachrichten folgen einem einheitlichen Format:

```
{
    "type": String,
    "payload": String
}
```

Je nach `type` ist der `payload` verschieden strukturiert. Für UI-Updates innerhalb einer Runde (`"UI_UPDATE"`) wird ein vollständiger Game-Snapshot gebroadcastet. Wird eine Runde beendet, so werden die Spielergebnisse der Runde ausgegeben.

## 3.5 Weitere Hinweise zur Implementierung

Insgesamt wurden 3 REST-APIs in dem Projekt implementiert. Die Dokumentationen befinden sich im Ordner `documentation/open-api-documentation`.

# 4 Lessons Learnt

Das Projekt hat uns ein gesamtes Semester begleitet und jedem Teammitglied die Möglichkeit gegeben, sich durch seine jeweiligen Aufgaben weiterzuentwickeln. Da jeder an Themen gearbeitet hat, die ihn tatsächlich interessiert haben, war es gut möglich, die Motivation über den gesamten Zeitraum aufrechtzuerhalten. Dennoch gibt es einige Punkte, die einer kritischen Reflexion bedürfen.

## 4.1 Zeitmanagement

In zukünftigen Projekten sollten wir den selbst gesetzten Zeitplan konsequenter einhalten. Idealerweise sollte dieser nicht nur grobe Meilensteine umfassen, sondern auch kleinere Zwischenziele. So lässt sich der Fortschritt besser einschätzen und zeitlichen Engpässen kann frühzeitig entgegengewirkt werden.

## 4.2 Kommunikation

Die Kommunikation im Team wurde durch regelmäßige Meetings sichergestellt. Dennoch kam es zu Absprachefehlern, die zu Bugs und fehlenden Features geführt haben. Da diese Probleme erst spät erkannt wurden, entstanden zusätzliche Verzögerungen im Projektverlauf. In Projekten der nächsten Semester kann man dies umgehen, indem man Arbeitspakete präziser formuliert und Aufgabengebiete eindeutig Personen zuordnet.

## 4.3 Auswahl der Technologien

Bei den meisten Services haben wir auf bereits bekannte Technologien gesetzt, was reibungslos funktioniert hat. Für zukünftige Projekte sollte im Voraus festgelegt werden, wo der Einsatz neuer Technologien sinnvoll ist, und dabei darauf geachtet werden, nicht den komplexesten Service als Experimentierfeld zu wählen. Darüber hinaus sollten Entwicklungsumgebung und Projektkonfiguration gemeinsam und frühzeitig eingerichtet werden, um spätere Probleme zu vermeiden.